Welcome back!

Go Concurrency

Go Recall

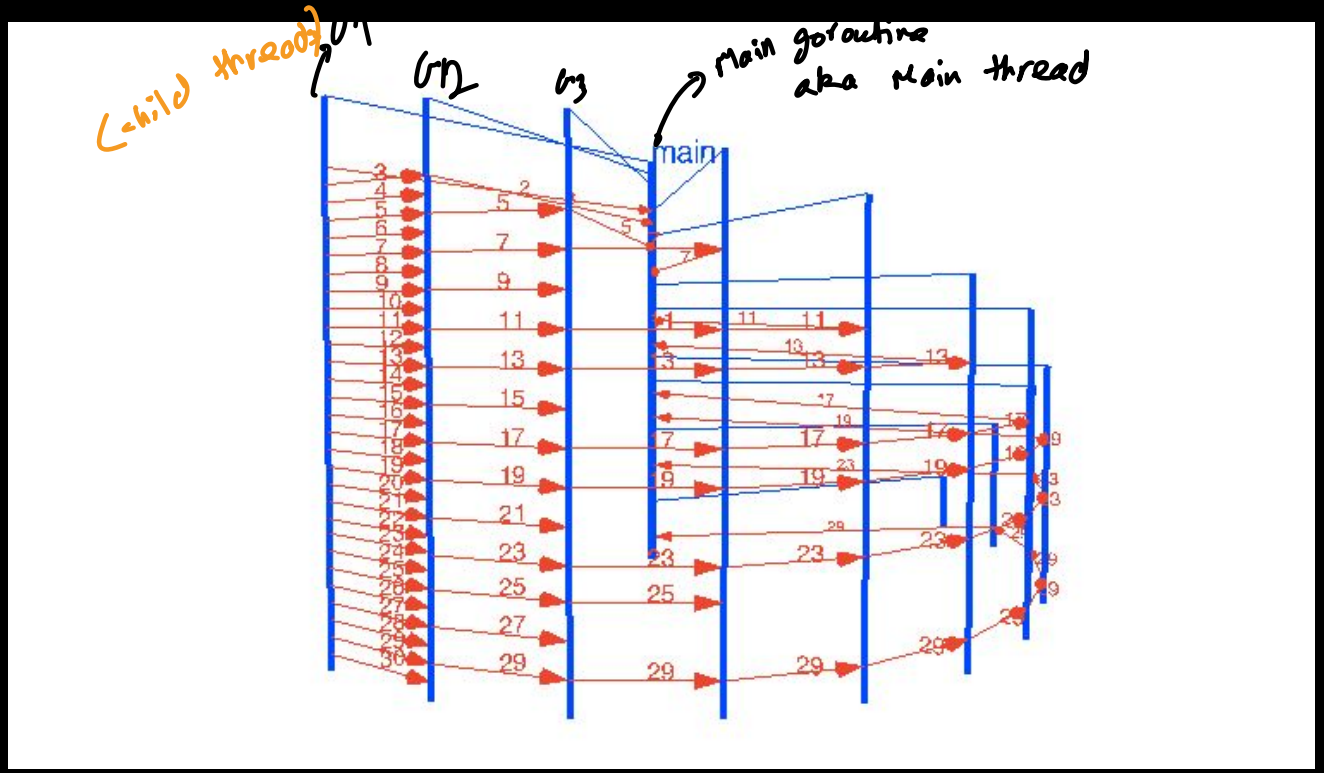
- Go Fundamentals
- Goroutines

Go Concurrency

- Goroutines
- Wait groups
- Mutex
- Channels
- Buffered Channels
- Select

- Race condition
- Deadlock

Go Concurrency



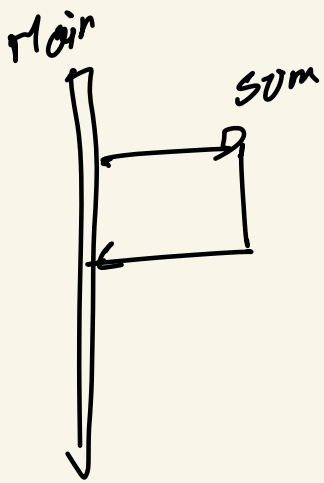
Concurrency Go vs Node.js

- Is Go Concurrent?
- Is Node.js Concurrent?

	Go	Nodejs
C	✓	v/s ✓
P	✓	✗

Go routine

- light weight
- Non-Det
- concurrency.



go func.

```
main() {
    go
Goroutine( res = sum(0, 6)

}
```

(2)

Go concurrency → give up CPU

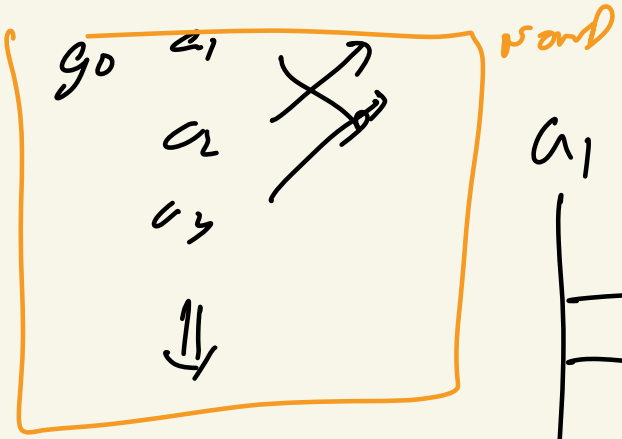
Nodejs ——— → never ———

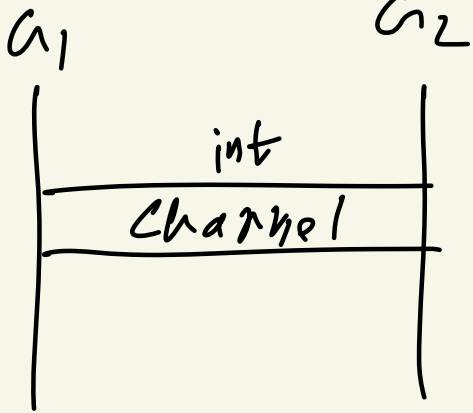
(3) channels:

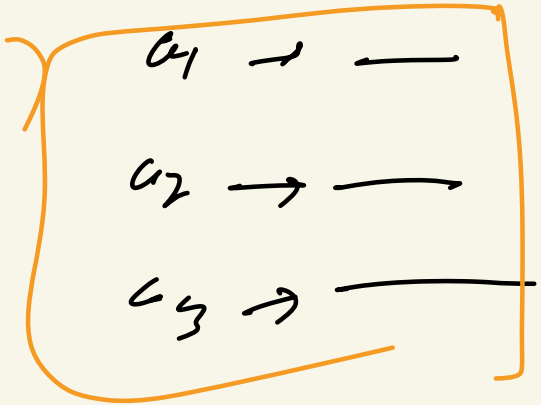
Channels

↳ Synchron.

↳ make







→ Detev.

```
func greet (ch)
{
    ch <- 3 ]  ①   S  ≡  send Block
}

main(){
    ch := make()
    go Greet( ch)
    val <- ch ]  ②   R ≡  R  Block
}
```

Concurrency Go vs Node.js

- Preemptive Scheduling
- Cooperative Scheduling

Cooperative Scheduling

- Deterministic Behaviour
- Never yield
  - *never gives up CPU*

Why Go choose Preemptive Scheduling?

- Preemptive scheduling gives **fairness and scalability**
- Fairness:
  - *No single goroutine can monopolize CPU*
  - *everyone eventually gets a chance to run.*

Concurrency Go vs Node.js

Preemptive Scheduling

- Scheduler can **interrupt a running goroutine at almost any point** and switch execution to another goroutine.
- goroutines can be stopped even during **CPU-heavy loops**
- Preemption makes systems **fair and responsive**, but introduces complexity.

Challenges

- Race Conditions
- You cannot assume **execution order** anymore: Non-deterministic
- **Context Switching** Overhead
- Deadlocks & Starvation

Example: Non-Deterministic

Channels

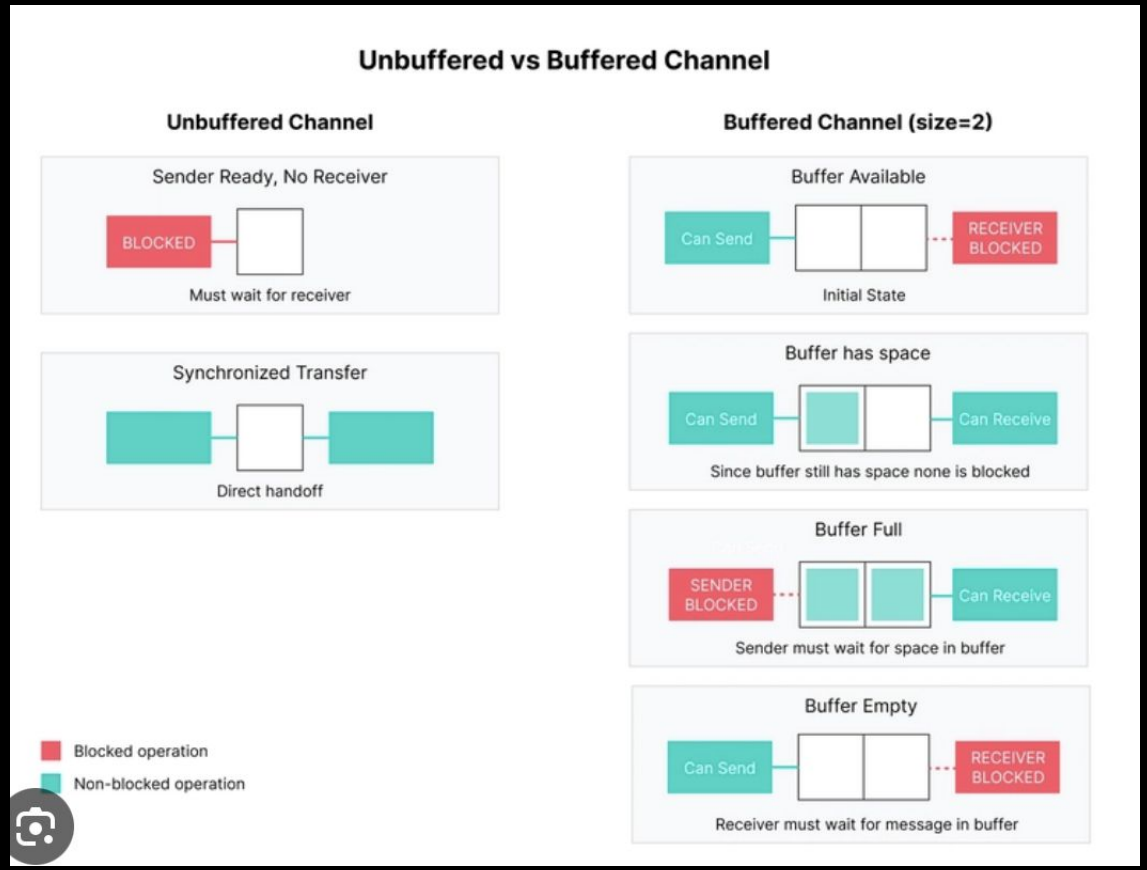
Buffered and Unbuffered

Channels

Channels are synchronization primitives

Channels

- Buffered channel
- Unbuffered channel



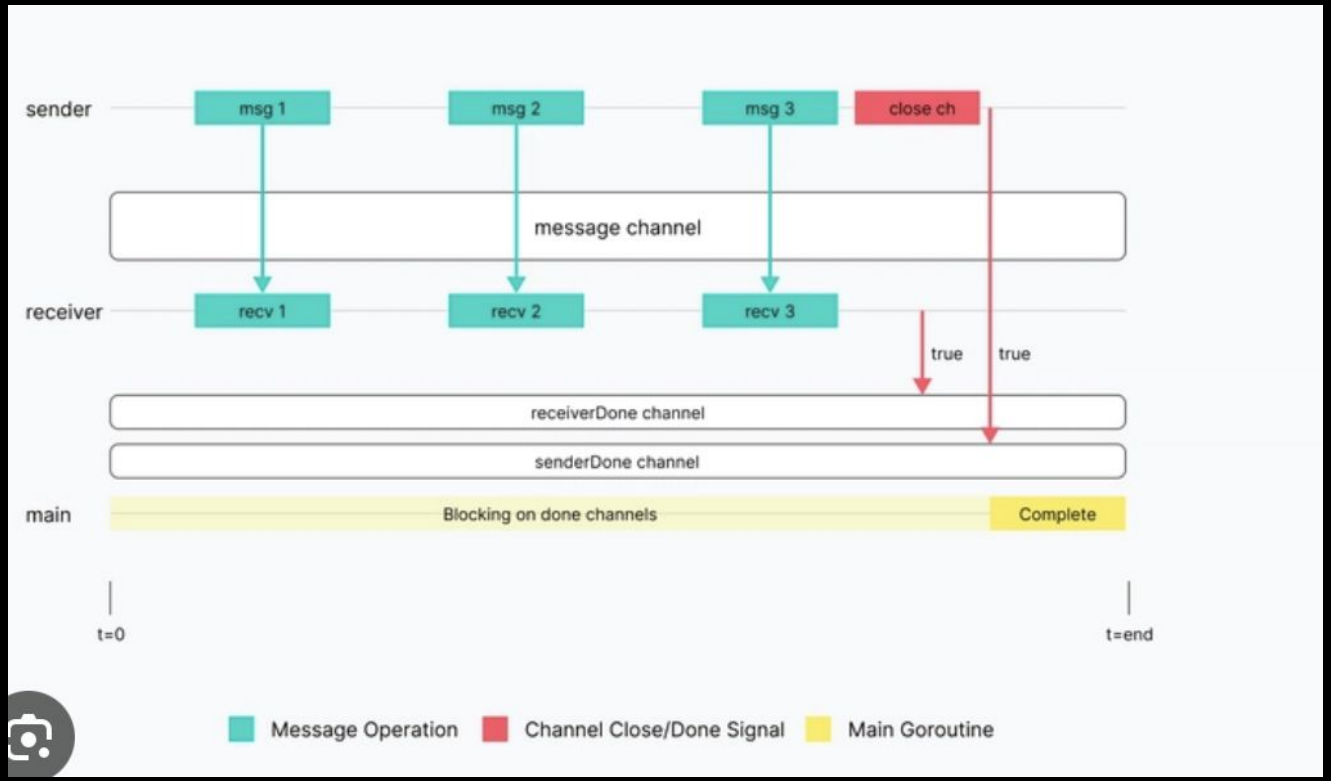
Buffered vs Unbuffered Channel

- Both allow multiple goroutines to run concurrently
- BUT they differ in when **synchronization happens**

By default

Aspect	Unbuffered	Buffered
Send behavior	Blocks immediately	Blocks only if full
Sync type	Immediate	Deferred (delayed)
Coupling	Tight	Loose
Control	Strong	Flexible

Channel


sender
msg 1
msg 2
msg 3
close ch
message channel
receiver
recv 1
recv 2
recv 3
true
true
receiverDone channel
senderDone channel
main
Blocking on done channels
Complete
t=0
t=end
Message Operation
Channel Close/Done Signal
Main Goroutine

Interview Questions

1. Sequential vs Parallel Execution
2. Buffered vs Unbuffered Channels
3. Is this parallel or sequential?

```go
for _, name := range names {
    go greetings(name, ch)
    <-ch
}
```

4. Will output always be A and then B?

```
go greetings("A", ch)
go greetings("B", ch)

<-ch
<-ch
```

Channels: Summary

Unbuffered = sync (handoff)

Buffered = queue (decoupling)

Unbuffered = synchronous coordination

Buffered = asynchronous coordination

Channels

- Channel Synchronization
- Buffered Channel
- Channel as a queue
- Channel Directions

Question: Ping Pong

Objective

Design a **simple message-passing system** using Go channels where:

- One function only sends a message
- Another function receives and forwards the message
- Channel directions must be enforced using compile-time restrictions

Question: Ping Pong

1. Create a function `ping`:
   - Accepts a **send-only channel**
   - Sends a given message into the channel
2. Create a function `pong`:
   - Accepts:
     - One **receive-only channel**
     - One **send-only channel**
   - Receives a message from the first channel
   - Sends the same message to the second channel
3. In `main`:
   - Create two buffered channels:
     - `pings`
     - `pongs`
   - Call `ping` to send a message
   - Call `pong` to transfer the message
   - Print the final output from `pongs`